CI/CD Flow Explanation (GitHub Actions → AWS ECS Fargate)

Overview

The CI/CD pipeline automates **build, test, containerization, deployment, and verification** of the Java Spring Boot application using **GitHub Actions**.

It supports:

- **Pull Requests** → validation only
- **Main branch** → full deployment to AWS

1️⃣ Pipeline Triggers

Pull Request (PR)

Triggered when:

- A pull request is opened or updated

Purpose:

- Validate code quality
- Prevent broken builds from reaching `main`

Actions:

- Build application
- Run unit tests
- (No deployment)

Push to `main`

Triggered when:

- Code is merged into the `main` branch

Purpose:

- Deploy the latest approved code to AWS

Actions:

- Build & test
- Build Docker image
- Push image to Amazon ECR
- Deploy to ECS Fargate

2️⃣ High-Level Flow

```
Developer
   |
   v
GitHub Push / PR
   |
   v
GitHub Actions Pipeline
   |
   +--> Build & Test
   |
   +--> Docker Build
   |
   +--> Push to ECR
   |
   +--> ECS Deployment (main only)
   |
   v
Application Live on ALB
```

Pipeline Stages

Stage 1: Build & Test

Purpose

Ensures the application compiles and tests pass before packaging or deployment.

Steps

1. Checkout repository
2. Set up Java (17)
3. Run Maven build:
4. `mvn clean verify`

Outcome

- Fails fast if tests or compilation break

- No AWS access required

Stage 2: Docker Image Build

Purpose

Creates an immutable, deployable artifact.

Steps

1. Build Docker image
2. Tag image with:
    - Git commit SHA (traceability)
    - `latest` (deployment default)

```
<account>.dkr.ecr.<region>.amazonaws.com/java-api:<sha>
<account>.dkr.ecr.<region>.amazonaws.com/java-api:latest
```

Stage 3: Push Image to Amazon ECR

Purpose

Stores container images securely in AWS.

Steps

1. Authenticate with AWS using GitHub Secrets
2. Login to ECR
3. Push both image tags

Security

- No long-lived credentials
- Secrets stored in GitHub encrypted secrets

Stage 4: Deploy to ECS (Main Branch Only)

🚫 This stage **does not run for pull requests**

Purpose

Deploys the new container image with zero downtime.

Steps

1. Register new ECS Task Definition revision
2. Update ECS Service
3. ECS performs **rolling deployment**
4. Old tasks are drained gracefully
5. New tasks pass ALB health checks

5️⃣ Deployment Verification

What Happens Automatically

- ECS waits for tasks to become healthy
- ALB health checks `/health`
- Deployment fails if:
  - Tasks crash
  - Health checks fail
  - Timeouts occur

Success Criteria

- Desired number of tasks running
- ALB reports healthy targets

6️⃣ Observability & Feedback

CloudWatch

- Container startup logs
- Runtime errors
- Deployment failures

New Relic

- Deployment markers
- Performance comparison before/after release
- Error rate tracking
- Alerting if thresholds exceeded

7️⃣ Environment Promotion Strategy

Environment	Trigger	Behavior
dev	Push to `main`	Automatic deployment
prod	Manual approval (optional)	Controlled release

Can be extended with GitHub Environments for approvals.

8️⃣ Rollback Strategy

Automatic

- ECS keeps previous task definitions
- Rolling deployments ensure availability

Manual Rollback

```
aws ecs update-service \
  --cluster <cluster> \
  --service <service> \
  --task-definition <previous-revision>
```

9️⃣ Security Built into CI/CD

✔️ Secrets stored securely in GitHub
✔️ No credentials committed to source control
✔️ IAM roles with least privilege
✔️ Private ECS tasks
✔️ Immutable artifacts

CI/CD Summary

Phase	What Happens
PR	Build + test only
Merge to main	Build → Docker → ECR → ECS
Deployment	Rolling, zero-downtime
Observability	Logs + metrics + alerts
Rollback	Immediate & safe
